Brain Neuron Digital Twin

An interactive digital twin of a modelled neuron: a 3D structure, a live signal animation and an ML prediction panel in one Google Colab notebook.

Run it in Colab: open the notebook

What it does

Part	Description
Simulator	A leaky integrate-and-fire (LIF) neuron generates a labelled dataset of 5,000 runs.
ML model	A neural network and a random forest predict the firing rate from synaptic input.
3D neuron	A procedurally grown neuron (dendrites, soma, branching axon) drawn with Plotly.
Signal animation	Excitatory inputs travel up the dendrites to the soma, then a spike travels down the axon.
Interactive panel	Sliders for excitatory synapses, inhibitory synapses and input rate. The panel shows the model's prediction next to a live simulation and a membrane-potential plot.

Quick start

- Open the Colab notebook: click here.
- Choose **Runtime, Run all**.
- Press **Play** under the 3D animation and move the sliders in the last panel.
- To run locally: `pip install numpy pandas scikit-learn matplotlib plotly ipywidgets jupyter`

If `neuron_lif_dataset.csv` is in the same folder as the notebook it is loaded. Otherwise the notebook generates an identical dataset (fixed random seed).

How it works

1. Neuron model (leaky integrate-and-fire)

Each millisecond the membrane potential V updates as follows. Excitatory and inhibitory events arrive as Poisson processes at (synapses x rate / 1000) events per millisecond. When V reaches the threshold, the neuron fires, resets, and stays silent for a 4 ms refractory period.

```
V = V - (V - V_rest) / tau + w_E * (excitatory events) - w_I * (inhibitory events)
```

Parameter	Value
Resting potential	-70 mV
Threshold	-50 mV
Time constant (tau)	20 ms
Excitatory weight (w_E)	1.0 mV
Inhibitory weight (w_I)	1.5 mV

Parameter	Value
Refractory period	4 ms

2. Dataset

`neuron_lif_dataset.csv` has 5,000 rows. Each row is one 500 ms simulation with randomly drawn inputs.

Column	Meaning
`n_exc`	Number of excitatory synapses (0 to 80)
`n_inh`	Number of inhibitory synapses (0 to 40)
`rate_hz`	Input rate per synapse in Hz (1 to 50)
`net_drive`	Engineered feature: max(0, n_exc - 1.5 x n_inh) x rate_hz / 4000
`firing_rate_hz`	Target: simulated output firing rate
`spiked`	1 if the neuron fired at least once, otherwise 0

About 34% of rows have at least one spike, so the data covers both silent and firing regimes. Firing rates range from 0 to 96 Hz.

3. Prediction model

Features: n_exc, n_inh, rate_hz, net_drive. Target: firing_rate_hz. Split: 80% train, 20% test.

Model	R^2 (test)	MAE (test)
MLP, hidden layers (32, 16), standardised inputs	0.988	0.92 Hz
Random forest, 200 trees	0.986	0.98 Hz

The net_drive feature gives the models a strong hint about the threshold behaviour. Removing it makes the task harder and is a good experiment.

4. 3D neuron and animation

The neuron is grown with a seeded recursive branching algorithm: 7 primary dendrites branching 4 levels deep, plus an axon with collaterals and a terminal arbor. The default seed gives 332 segments and 168 dendritic tips.

The animation tracks each segment's path distance from the soma. Pulses are Gaussian bumps travelling along that coordinate: pink pulses run from dendritic tips to the soma, then an amber spike runs outward along the axon.

Repository contents

```
neuron_digital_twin.ipynb   full notebook (run in Google Colab)
neuron_lif_dataset.csv      simulated dataset (5,000 rows)
README.md                   project description
```

Limitations

- The data is **simulated**, not recorded from real neurons.

- The twin models a simplified point neuron, so the 3D shape does not affect the simulation. It is used for visualisation.
- The model learns the behaviour of this specific LIF configuration and will not generalise to other neuron types without retraining.

Ideas for next steps

- Load a real reconstructed morphology from NeuroMorpho.org (SWC files).
- Use a multi-compartment model so the dendritic structure affects the output.
- Predict spike timing, not just firing rate.
- Remove the net_drive feature and compare model performance.

Tech stack

Python, NumPy, pandas, scikit-learn, Matplotlib, Plotly, ipywidgets